

CAT-SOOP Course Analytics — Architecture

COURSE

Software Tools and Techniques for CSE · IIT Gandhinagar

BUILDS ON

Project Plan (Submission 1)

SUBMISSION

2 — Architecture and design

Summary

We add an instructor analytics dashboard to CAT-SOOP **without modifying CAT-SOOP's code**. A nightly Python job reads student activity through CAT-SOOP's own functions, computes summaries and stores them in PostgreSQL. The dashboard is a set of ordinary CAT-SOOP course pages that read only those summaries. CAT-SOOP's own login and permission system decides who can see them.

The design rests on one rule: **heavy work happens offline; pages only read precomputed results**. That rule is what keeps the live course safe for students.

QUESTION 1

Architecture and design decisions

Overview

The system has two halves that share only a database:

- **Batch pipeline (offline):** extract → normalise → validate → load → compute summaries. It runs nightly from a scheduled job.
- **Dashboard (inside CAT-SOOP):** course pages, a course handler and a small plugin. They are served by CAT-SOOP's existing web processes.

CAT-SOOP remains the source of truth. If the analytics database were deleted, re-running the job would rebuild it.

Key design decisions

#	DECISION	WHY	REJECTED ALTERNATIVE
D1	Build the dashboard as CAT-SOOP course pages , not a separate web app	Login, roles, per-course permissions and site styling come for free	FastAPI + React (plan v1.0): needs its own login and a second copy of "who is staff"
D2	Compute offline, display precomputed summaries	Pages stay fast at any class size and never compete with student submissions	Computing on page load: CAT-SOOP's existing staff views already do this and are slow at 250 students
D3	Read CAT-SOOP data only through its Python API (<code>cslog</code> , <code>csqueue</code> , <code>user</code>), in one adapter module	Logs are pickled, can be encrypted, and may live in files, MongoDB or Firestore	Parsing log files directly: breaks under encryption or other storage backends
D4	Record page visits with a course plugin on the <code>post_handle</code> hook , once per student, page and session	Visits aren't logged today; this hook fires only for real page views	<code>post_render</code> : also fires on 404 and error pages
D5	Protect every analytics page with one inherited handler that checks permission first	<code>preload.py</code> runs before login, so it cannot check permissions itself	Hiding the menu link: not security
D6	Gate access on CAT-SOOP's existing <code>whdw</code> permission (configurable)	No separate staff list that could drift out of sync	Hard-coding role names
D7	PostgreSQL , with a read-only database role for the web pages	That role can only read summary tables, a second barrier behind D5	SQLite: no per-role privileges
D8	Incremental, idempotent extraction (per-log watermarks + upserts)	Nightly cost grows with new activity, not total history; re-runs are safe	Full re-scan every night

#	DECISION	WHY	REJECTED ALTERNATIVE
D9	Rule-based attention flags, each with its stated reason	Explainable; there is no labelled data to train a model	Machine-learning risk score
D10	Discard answer text during extraction; hide groups under 5 students	Metrics don't need the answers; minimises privacy risk	Storing full activity records

Tools and frameworks

AREA	CHOICE
Language	Python 3.10+ (required: CAT-SOOP data is only readable from Python)
Host platform	CAT-SOOP (this fork, pinned commit): plugins, course handlers, templates
Database	PostgreSQL 16 via <code>psycopg3</code>
Front end	Server-rendered HTML, vanilla JS, Chart.js 4 (bundled locally, no CDN)
Scheduling	systemd timer or cron, nightly
Testing	pytest, a synthetic persona course, Locust for load tests
Quality and CI	ruff, import-linter (enforces module boundaries), GitHub Actions

QUESTION 2

Requirements the architecture focuses on

Functional

ID	REQUIREMENT
F1	Course overview: students, active students, average score, completion per module
F2	Module progress that separates opened , started and completed
F3	Question analytics: first-attempt success, attempts to succeed, abandonment
F4	Individual student progress compared with the class
F5	Attention list with reasons; difficulty and flawed-question detection
F6	Staff-only access; role-based landing pages for guest, student and staff
F7	Nightly, incremental data sync; multi-course and multi-semester support

Quality attributes

ATTRIBUTE	TARGET
Performance	Any dashboard page responds in < 300 ms at 5,000 students; the visit hook adds < 5 ms
Non-interference	Student submissions are unaffected by dashboard use or by extraction
Security	Students and guests are refused on every URL, including data endpoints; permission is per course
Privacy	No answer text stored; no statistic shown for groups under 5
Correctness	Attempt counts match CAT-SOOP's own counts exactly; ungraded $\neq$ zero
Reliability	Analytics failures never affect the course; the last good data stays visible
Maintainability	Zero changes to <code>catsoop/</code> ; CAT-SOOP-specific code lives in one module
Extensibility	A new page or course is added by adding a folder, not by changing existing code

Constraints

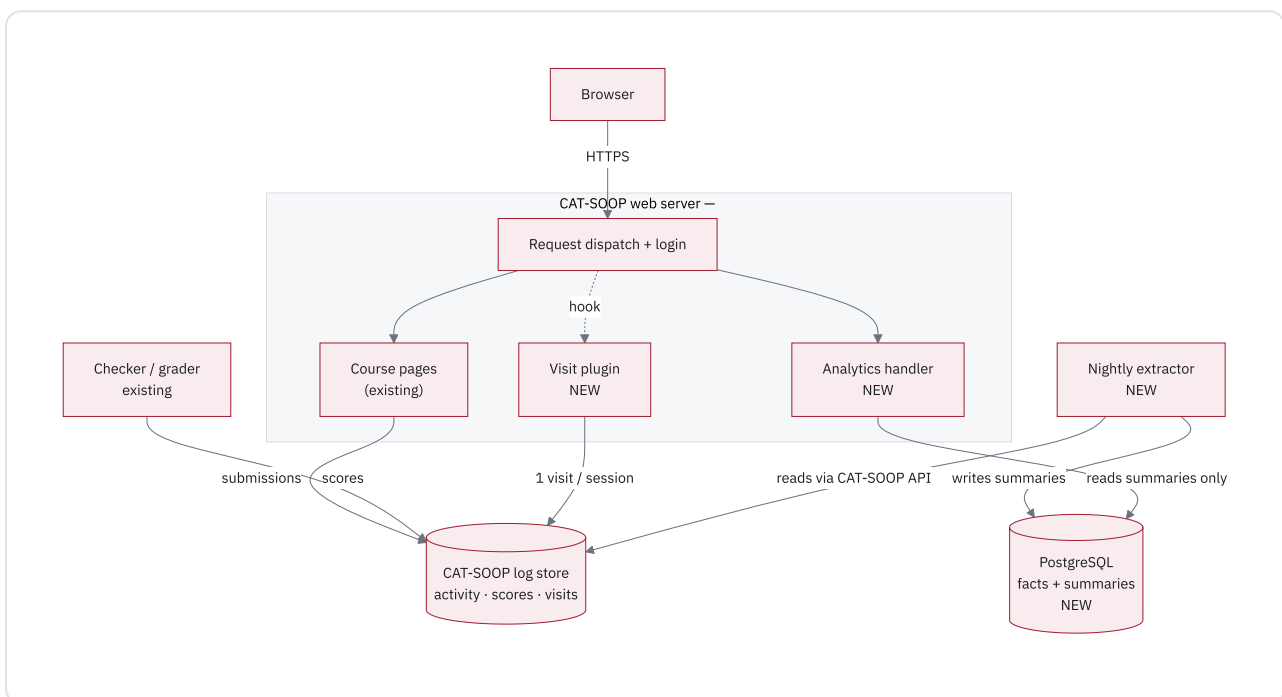
- CAT-SOOP's source must not be modified.
- Data can only be read with Python, through CAT-SOOP's own modules.
- The dashboard runs inside CAT-SOOP's web processes: no separate server, no frontend build step.

QUESTION 3

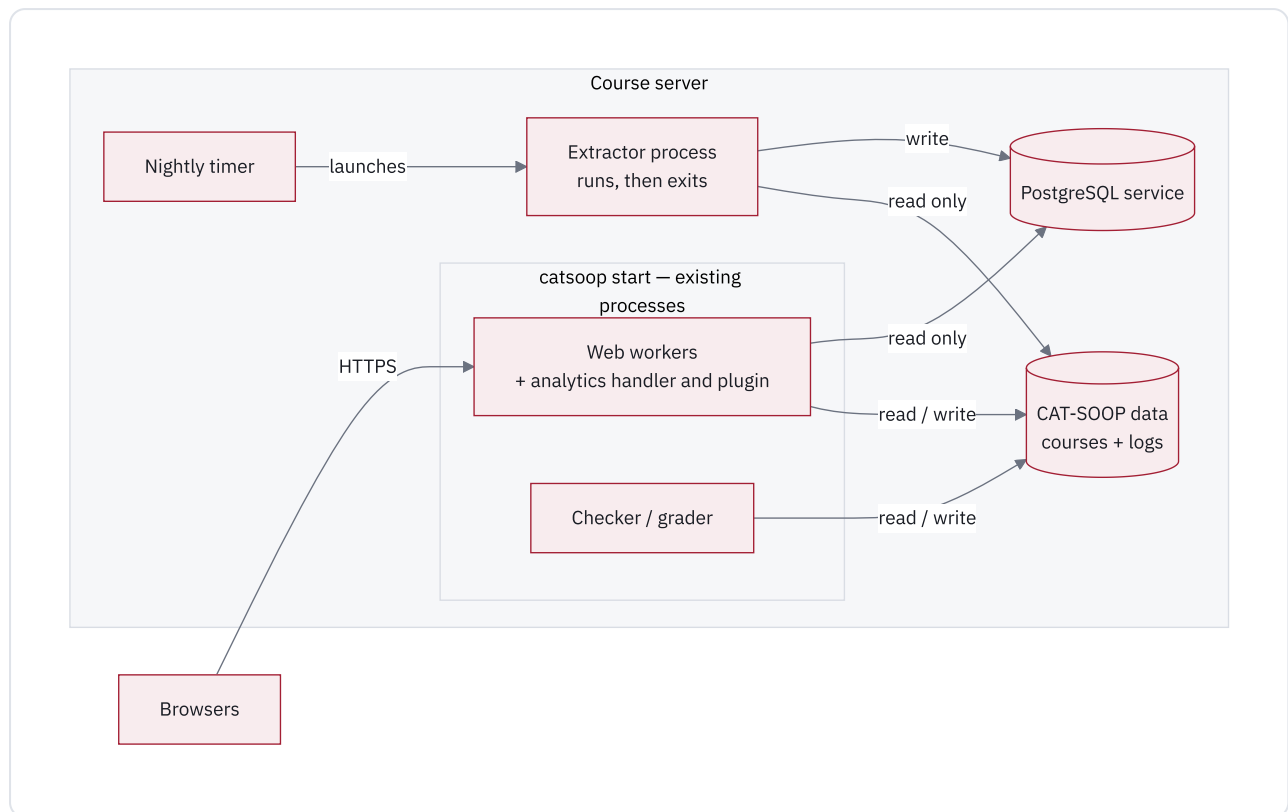
Overall system structure

Components and how they interact

Browsers reach CAT-SOOP's existing web server, where our **visit plugin** and **analytics handler** run. Student activity, grader scores and visits go into CAT-SOOP's log store. The **nightly extractor** reads that store and writes PostgreSQL; the handler only reads PostgreSQL.



Run-time processes



PROCESS	STARTED BY	ROLE
CAT-SOOP web workers	catsoop start	Serve course and analytics pages; our handler and plugin run inside them
Checker / grader	catsoop start	Grades submissions in the background (existing)
Extractor	Nightly timer	Reads logs, computes summaries, exits
PostgreSQL	System service	Stores analytics data

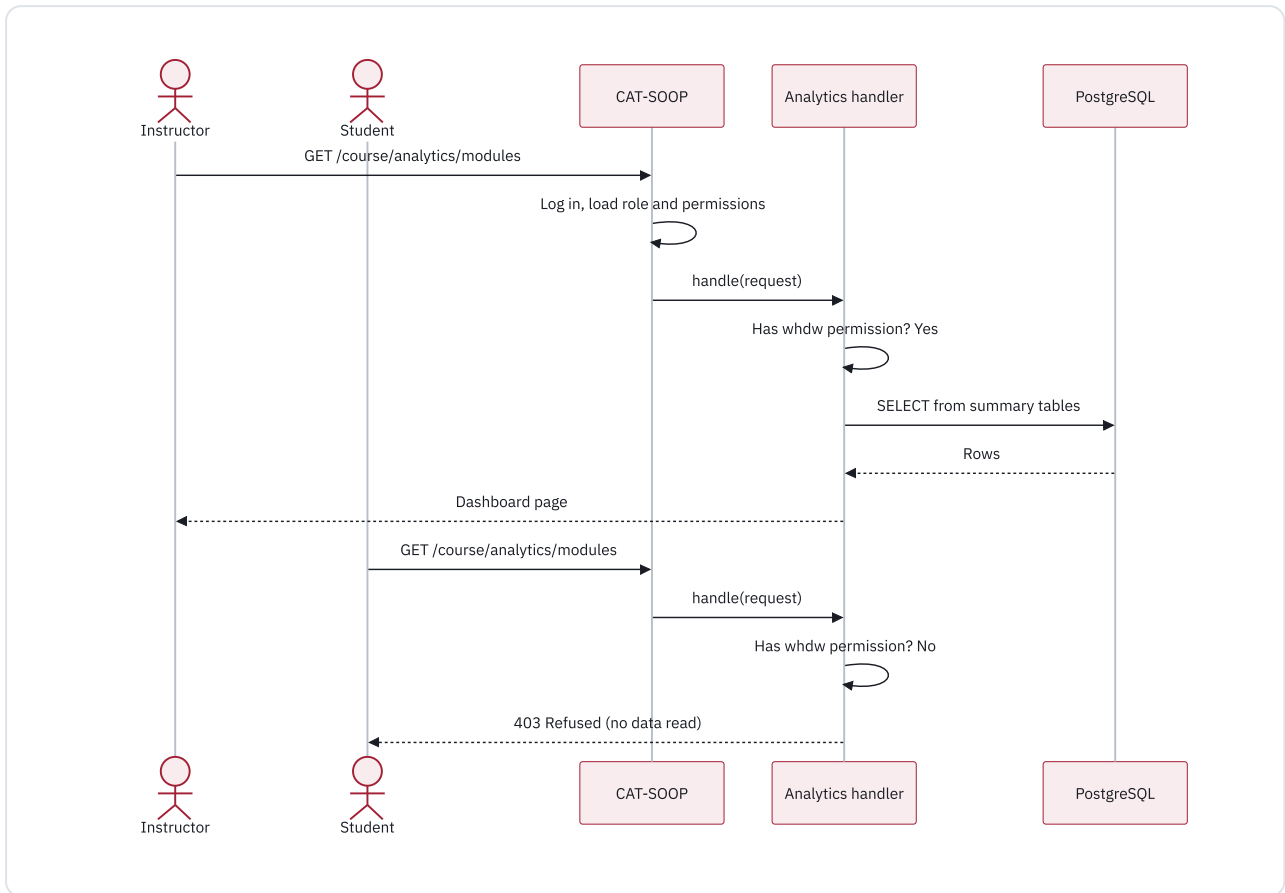
The extractor writes each run in **one transaction** and publishes it atomically, so pages never see half-updated numbers. A database lock prevents two runs from overlapping.

Interfaces

INTERFACE	USED BY	CONTRACT
Dashboard pages	Instructor's browser	<code>GET /<course>/analytics</code> plus <code>/modules</code> , <code>/questions</code> , <code>/students</code> , <code>/student?u=<pseudonym></code> , <code>/attention</code> → HTML page
Data endpoints	Dashboard JavaScript	Same URLs with <code>?format=json&sort=&page=</code> → <code>{"as_of": "<ISO time>", "rows": [...], "next": <page>}</code> ; <code>?format=csv</code> for export
Course handler	CAT-SOOP dispatch	<code>handle(context) -> str (page) or (status, headers, body)</code> (JSON, CSV, 403)
Plugin hooks	CAT-SOOP dispatch	<code>post_handle.py</code> (record visit) and <code>post_auth.py</code> (staff menu link), run with the request context: <code>cs_user_info</code> , <code>cs_path_info</code> , <code>cs_session_data</code>
CAT-SOOP API	Extractor, plugin	<code>cslog.read_log(user, path, log)</code> , <code>cslog.update_log(user, path, log, entry)</code> , <code>csqueue.get_results(job_id)</code> , <code>user.list_all_users(context, course)</code>
Visit log entry	Plugin → extractor	<code>{"t": "<timestamp>", "sid_h": "<hashed session id>"}</code> in log <code>analytics_visits</code>
Command line	Timer, operators	<code>catsoop-analytics extract --course C [--full] · rebuild · verify · migrate</code> — exit code + JSON run report
Database	Extractor, handler	Writer role: all tables. Reader role: <code>SELECT</code> on summary tables only

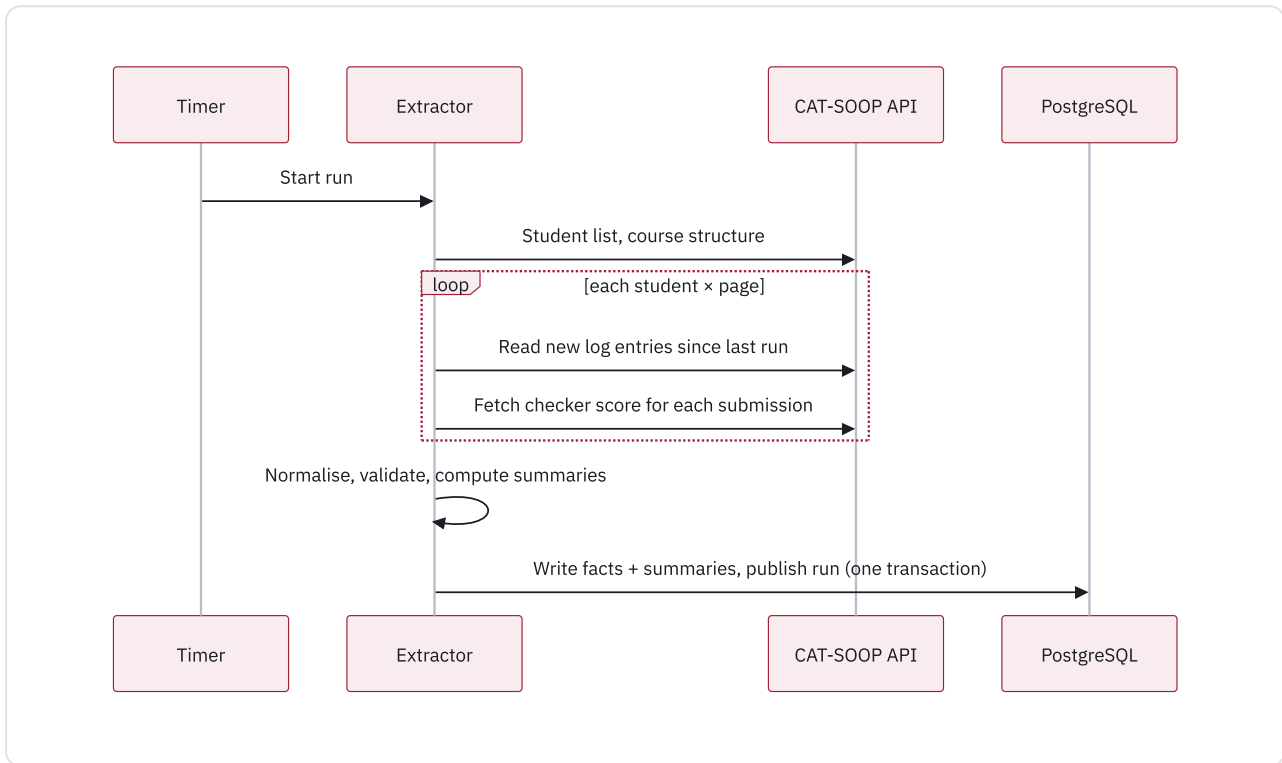
Instructor opens the dashboard; a student tries the same URL

CAT-SOOP logs the user in first. The analytics handler then checks the `whdw` permission **before** any query: an instructor gets summary rows, a student gets 403 and no data is read.



Nightly extraction

The timer starts the extractor. It fetches the roster, course structure, new log entries and checker scores through CAT-SOOP's API, then normalises and validates them, computes summaries, and writes and publishes everything in one transaction.



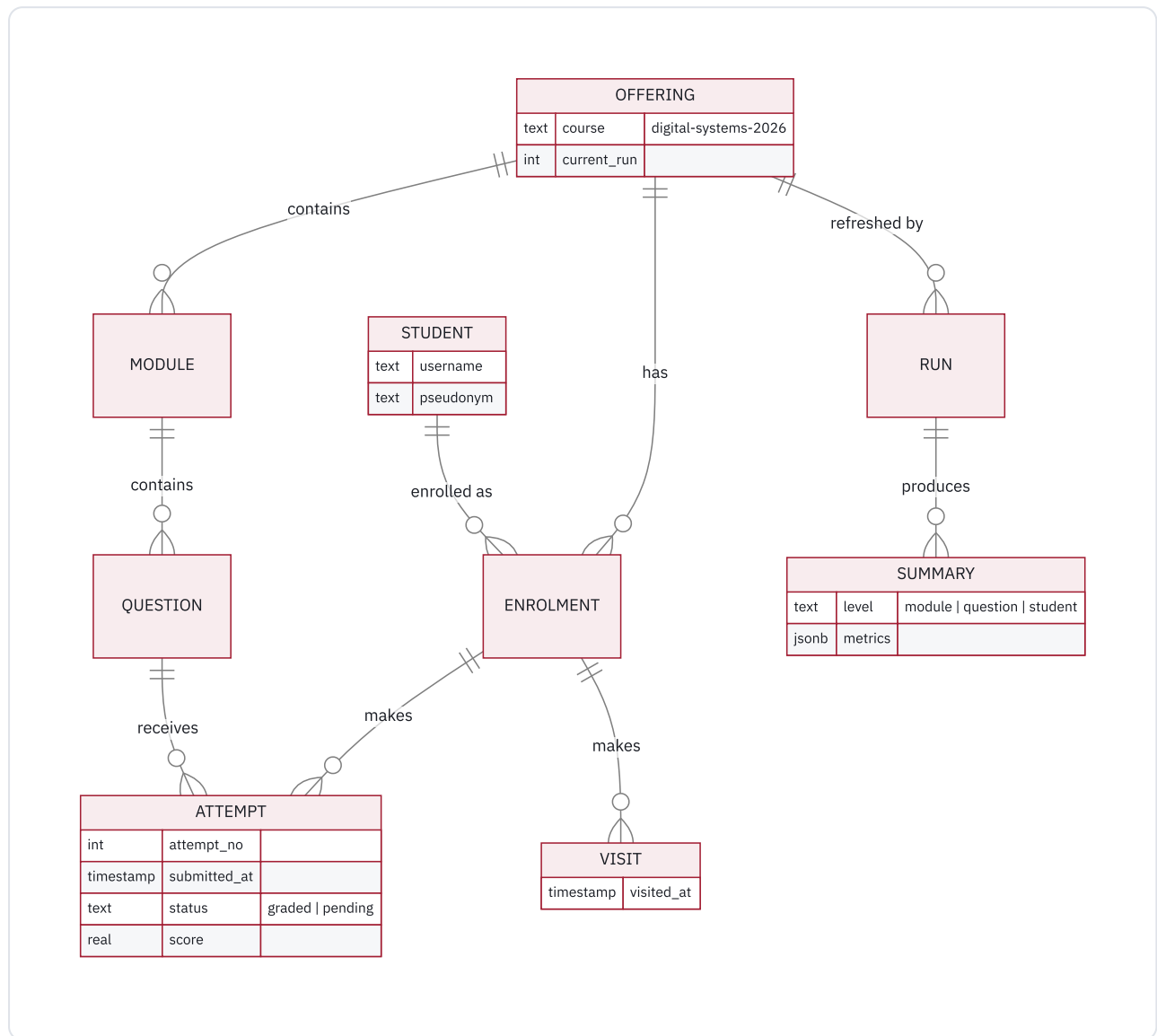
Data structures

What CAT-SOOP already stores (input):

LOG	CONTAINS	USED FOR
<code>problemactions</code>	Every submit / check / save, with timestamp and grader job id	Attempt history
Checker results	Score for each grader job	Attempt scores
<code>problemstate</code>	Current state only (overwritten each time)	Cross-checking counts
<code>__USERS__/*.py</code>	Each user's role and section	Roster
<code>analytics_visits</code> (new)	One entry per student, page and session	"Opened a module"

Analytics database (output):

A course **offering** contains modules and questions; students **enrol** in an offering; **attempts** and **visits** belong to an enrolment; each extraction **run** produces the **summaries** that pages read.



Facts (attempts, visits) can always be rebuilt from CAT-SOOP. *Summaries* are tied to a run, so pages always read one consistent snapshot.

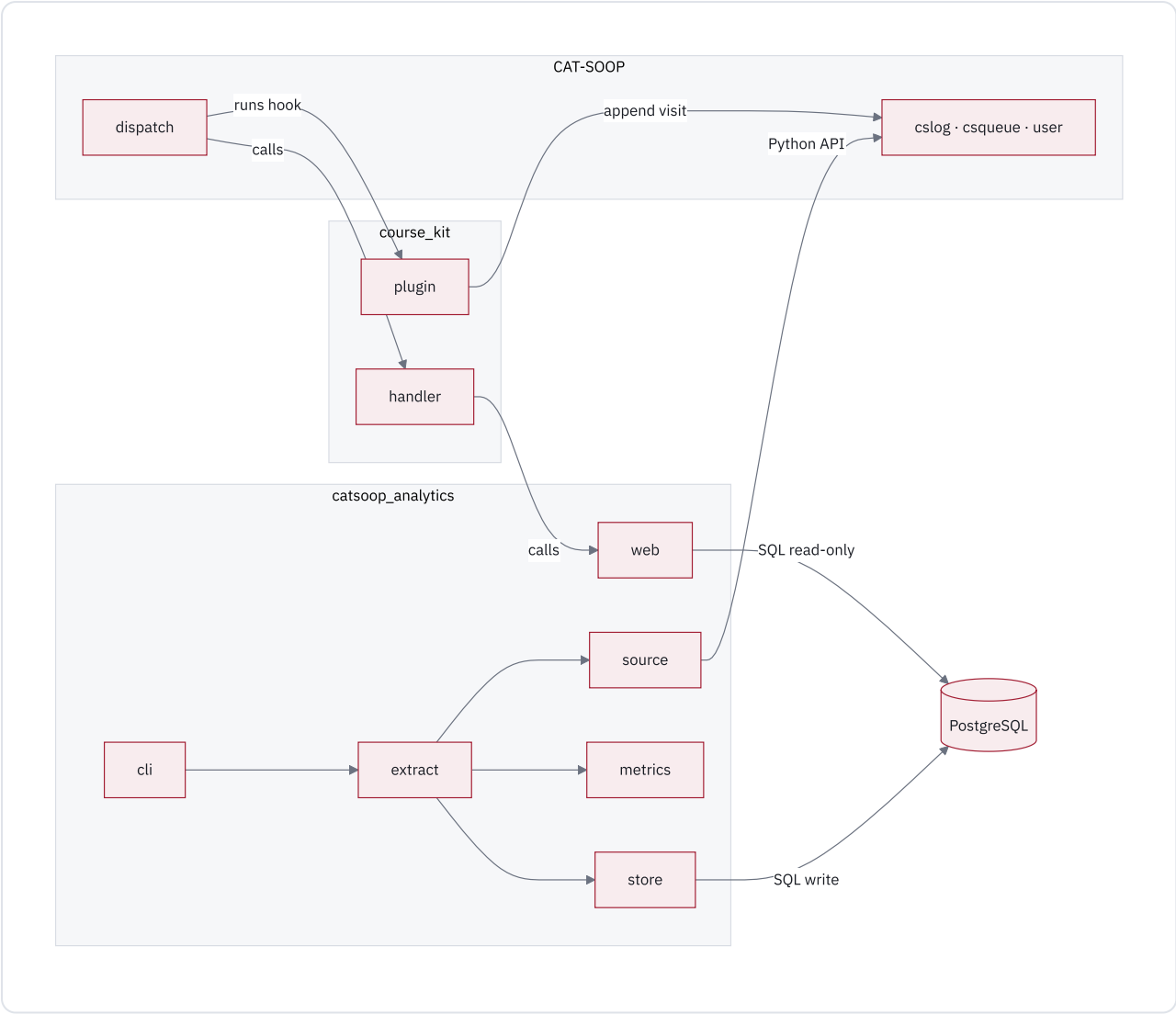
QUESTION 4

Major code modules and how they communicate

Layout

```
catsoop/                                unchanged
analytics/
├─ catsoop_analytics/                   installable Python package
│   ├─ source/                         the only code that imports catsoop
│   ├─ extract/                       normalise, validate, run orchestration
│   ├─ metrics/                      progress, difficulty, attention (pure functions)
│   ├─ store/                        database access and migrations
│   ├─ web/                          access check, read-only queries, page rendering
│   └─ cli.py                        extract · rebuild · verify · migrate
└─ course_kit/                        copied into each course folder
    ├─ __PLUGINS__/analytics/         visit capture, staff menu link
    ├─ __HANDLERS__/analytics/        permission check → web package
    └─ analytics/                    dashboard pages + static JS/CSS
```

Module interaction



FROM → TO	COMMUNICATION
CAT-SOOP → plugin, handler	In-process calls at defined extension points
Plugin → CAT-SOOP log store	<code>cslog.update_log</code> (one append)
Handler → <code>web</code>	Python function call
<code>web</code> → PostgreSQL	SQL <code>SELECT</code> on summary tables, read-only role
Browser → handler	HTTPS; <code>?format=json</code> for chart data
<code>extract</code> → <code>source</code> → CAT-SOOP	Python calls returning plain dataclasses
<code>extract</code> → <code>metrics</code> → <code>store</code>	Function calls; one SQL transaction per run

Boundaries enforced in CI: only `source` imports CAT-SOOP; `metrics` has no I/O; `web` can only read.

What each module provides

MODULE	MAIN FUNCTIONS	RETURNS
<code>source</code>	<code>students(course)</code> , <code>modules(course)</code> , <code>log_entries(user, page, log)</code> , <code>checker_score(job_id)</code>	Plain dataclasses; answer text already removed
<code>extract</code>	<code>run(course, full=False)</code> , <code>to_events(raw_entries)</code> , <code>validate(events)</code>	Attempts, visits, anomalies, run report
<code>metrics</code>	<code>progress(facts)</code> , <code>difficulty(facts)</code> , <code>attention(facts, rules)</code>	Summary rows (pure functions, no I/O)
<code>store</code>	<code>begin_run()</code> , <code>upsert_facts()</code> , <code>write_summaries(run)</code> , <code>publish(run)</code> , <code>reader()</code>	Database transactions and connections
<code>web</code>	<code>require_staff(context)</code> , <code>route(context)</code> , <code>overview()</code> , <code>modules()</code> , <code>questions()</code> , <code>student()</code> , <code>attention()</code>	HTML fragments or JSON
<code>cli</code>	<code>extract</code> , <code>rebuild</code> , <code>verify</code> , <code>migrate</code>	Exit code + run report

QUESTION 5

How the design satisfies the requirements

Functional requirements

ID	DELIVERED BY	DECISIONS
F1	<code>metrics.progress</code> computes module and course summaries; the Overview page reads them	D2
F2	The visit plugin supplies <i>opened</i> ; attempts supply <i>started</i> and <i>completed</i> ; <code>metrics.progress</code> keeps all three separate	D4, D8
F3	Attempt history joined with checker scores in <code>extract</code> ; <code>metrics.difficulty</code> computes success rates and abandonment	D3, D8
F4	Student summaries, shown at <code>/analytics/student?u=<pseudonym></code> next to class averages	D2, D10
F5	<code>metrics.attention</code> rule functions with stated reasons; ability bands and flawed-question check in <code>metrics.difficulty</code>	D9
F6	Inherited analytics handler checks <code>whdw</code> ; course home page shows different content per role	D1, D5, D6
F7	Nightly timer and watermarks; all data keyed by course offering; each course gets its own copy of <code>course_kit</code>	D8

Quality attributes

REQUIREMENT	HOW THE STRUCTURE AND DECISIONS SATISFY IT
Performance	Pages read small precomputed tables with indexed, paginated queries (D2); the visit hook is one append per session (D4)
Non-interference	Heavy reading runs in a separate process at night (D2); CAT-SOOP never depends on the analytics database
Security	CAT-SOOP authenticates first; one inherited handler checks permission before any data access (D5, D6); the database role blocks raw data (D7)
Per-course access	Permissions come from each course's own user files, so staff on one course get nothing on another (D1)
Privacy	Answer text is dropped in the adapter; small groups are suppressed before any page or export sees them (D10)
Correctness	Attempt = <code>submit</code> only; ungraded submissions stay <i>pending</i> ; a <code>verify</code> command diffs our counts against CAT-SOOP's
Reliability	Atomic publication keeps the last good run visible; failed runs roll back and retry safely (D8)
Maintainability	Only CAT-SOOP extension points are used; all CAT-SOOP-specific code is in <code>source/</code> (D3)
Extensibility	New page = new folder, automatically protected (D5); new course = copy <code>course_kit/</code> ; new rule = one function (D9)
Scalability	Incremental runs scale with new activity (D8); all data is partitioned by course offering

Trade-offs we accept

- **Data is up to a day old.** Each page shows when its data was last refreshed; runs can go hourly if needed.
- **No modern front-end framework.** Server-rendered tables and a small charting library are enough, and match the platform.
- **We rely on internal CAT-SOOP functions.** They are isolated in one module and covered by contract tests.

Appendix: What the CAT-SOOP source confirmed

We read the source before designing. These findings drive the decisions above and correct parts of Submission 1.

FINDING	WHERE	DECISION
Every submit/check/save is appended to <code>problemactions</code> ; scores for auto-graded questions arrive later from the checker	<code>default.py</code> <code>log_action</code> , <code>grader.py</code>	D3, D8
Page views are not logged; AJAX actions return before <code>post_handle</code> , but error pages still trigger <code>post_render</code>	<code>dispatch.py</code> L422, L876	D4
<code>preload.py</code> runs before login; courses can supply their own handler, which runs after	<code>loader.py</code> , <code>tutor.py</code> <code>handler()</code>	D5
<code>whdw</code> is CAT-SOOP's existing staff-monitoring permission	Course <code>preload.py</code> , <code>default.py</code>	D6
The log store can be files, MongoDB or Firestore, optionally encrypted	<code>cslog.py</code> <code>initialize()</code>	D3
URLs map to folders, so <code>/students/<id></code> would 404; use <code>/student?u=<id></code>	<code>dispatch.py</code> <code>is_resource</code>	Routes
Staff "view as student" actions are logged under the student's name	<code>auth.py</code>	Excluded from metrics